

Learn Mode (SRS) — техническая ВЫЖИМКА

Проект: Poker Trainer — веб-тренажёр покерных префлоп-диапазонов · **Блок:** Study / Learn Mode (интервальные повторения) · **Дата:** 2026-06-17

Прод-движок: `srs.py` + `srs_api.py` (SM-2) · **Эксперимент:** `srs_fsrs.py` (FSRS-lite, к API не подключён) · **Тесты:** 64 + 18 + 45 зелёных

Документ — выжимка для внешнего ревью. Описывает математику планирования, сознательные отклонения от канонического SM-2 и их обоснование, экспериментальный FSRS-параллель и дорожную карту.

1. Назначение и архитектура

Learn Mode тренирует **запоминание** диапазонов (в отличие от Practice Mode, где идёт розыгрыш). Одна **карточка** = одно решение в одном споте: кортеж (рука, позиция героя, спот, позиция оппонента), например `AKs · SB · vs_RFI · UTG`. «Ответ» карточки — распределение частот действий из тренируемого диапазона (`correct_strategy`, напр. `{"3bet": 0.6, "call": 0.4}`).

Состояние — JSON по файлу на пак (`srs_state/<pack>.srs.json`): cash и MTT не пересекаются в обучении. Ядро (`srs.py`) не имеет зависимостей кроме `stdlib`; алгоритм и `persistence` намеренно держатся в прозрачном, легко инспектируемом JSON.

Статус ядра: SM-2 — тестовый старт, дальше FSRS. Прод-движок намеренно на SM-2 — самый простой и быстрый в реализации вариант, с которого мы начали, чтобы запустить Learn Mode и начать копить реальные ответы. Это осознанная фаза холодного старта, а не отклонение: её предписывает сам первоисточник FSRS (KDD'22, Appendix A.2 — SM-2 на старте, обученная модель памяти после накопления данных). Следующий шаг — переход на FSRS: каркас FSRS-lite на `py-fsrs` (Open Spaced Repetition) уже собран параллельно и готов к подключению; переключение — после 2-3 недель логов и A/B против SM-2 (Track A.4). Настоящий документ — обоснование и карта этого перехода.

Learn Mode (SRS) — поток повтора и правила планирования

Poker Trainer · srs.py (SM-2, прод) + srs_fsr.py (FSRS-lite, эксперимент, не в проде)

GOOD AGAIN EASY dev-only

1 · Построение колоды (один раз на пак)

Range JSON

spots: RFI · vs_RFI · vs_3bet ...

init_cards_from_spots()

→ 169 пок, implicit fold = {fold:1.0}

srs_state/<pack>.srs.json

Card = (hand, position, spot, villain)

1 пак = 1 колода = 1 файл стейта
cash и MTT не пересекаются в обучении

2 · Ежедневная очередь — get_due_cards(today, new_limit)

due reviews: next_review ≤ today — отдаются всегда
+ new cards: md5(card_id)-шафл, срез по new_limit/день

Queue на сегодня

due + капнутые new

new_limit: 10 / 15 / 30 / 50 / ∞
кап показа, не свойство колоды
learned ≥ 21 дн → долговременная память

3 · Цикл одного повтора (review loop)

1 GET /api/srs/next → карточка-вопрос

UI: кнопки действий по типу спота + «Показать ответ»

2 Ответ → grade_answer(card, action, marked_easy)

action ∈ strategy (freq > 0) → GOOD · marked_easy → EASY

action ∉ strategy / неизвестно → AGAIN

«Показать ответ» (reveal=true) → AGAIN (принудительно)

3 update_card(rating) — пересчёт SM-2

interval & ease обновляются · запись в history (Δt, rating)

4 save_state → раскрыть correct_strategy

ответ показывается ПОСЛЕ грейда — вопрос не спойлерится

5 Следующая карта — auto-next 1.5 с

после reveal авто-перехода нет (изучаешь ответ вручную)

4 · SM-2 — эффект рейтинга на (interval, ease)

GOOD · 3

interval: 0→1→3→round(prev×ease)

лесенка graduation · ease без изменений

AGAIN · 1

interval → 1 · ease → 0.20

пол ease = 1.3 · streak обнуляется

EASY · 4

0→3, иначе round(prev×ease×1.3)

ease + 0.15 · из UI убрана (ядро живо)

HARD · 2

в Learn UI не используется

примитив SM-2 сохранён в ядре

classify(): dominant ≥ 0.95 → pure_<action>, иначе mixed
ease₀=2.5 · MIN_EASE=1.3 · NEW/день=15 · learned≥21д

5 · FSRS-lite — параллельно SM-2, НЕ подключён к API

seed_difficulty(strategy) → [1..10]

pure → 2.0 · near-pure(0.9) → ~4.4 · ровный микс → 8.0 · мульти-вей → clamp 10

FSRS-6 (py-fsrs)

desired_retention 0.95 · maximum_interval 120 · learning_steps ()

Зачем сид: при бинарном вводе Good не двигает ни ease, ни difficulty

→ pure-карту и близкий микс не различить без стартовой трудности

fsrs не в requirements.txt · A/B-харнесс гоняет историю из srs_state/*

Один повтор = HTTP-цикл /next → /answer. Грейдинг бинарный (Chessable-стиль): «попал в стратегию / нет», без самооценки сложности.

Рис. 1. Поток одного повтора и правила планирования: построение колоды → ежедневная очередь → цикл /next → /answer; справа — правила SM-2 и экспериментальный FSRS-lite.

2. Модель карточки и объективный грейдинг

Колода строится из spots range-файла функцией init_cards_from_spots(): каждая (позиция, спот, оппонент) с хотя бы одной явной записью **доразворачивается до всех 169 стартовых рук** — недостающие руки становятся pure-fold карточками {"fold": 1.0}. Это сознательно: Learn тренирует и «не забыть открыть», и «не забыть сбросить мусор».

Ввод **принудительно объективный** (Chessable/Duolingo-стиль): пользователь не самооценивает память, а кликает действие, и система объективно знает результат. Мост UI→рейтинг — grade_answer(card, user_action, marked_easy):

```
action ∈ correct_strategy и freq > 0 → GOOD (или EASY, если marked_easy)
action ∉ correct_strategy / неизвестно → AGAIN
«Показать ответ» (reveal=true) → AGAIN (принудительно, в /answer)
```

Микс-карточки. Для A5s UTG = {open: 0.7, fold: 0.3} **оба** действия (open и fold) грейдятся как GOOD. Это знание о микс-природе спота, а не угадывание броска ГСЧ — сознательное решение #7. «Правильно» = членство в стратегии, а не совпадение с конкретным розыгрышем.

Третья кнопка «Показать ответ» (решение #6, поправка 2026-06-01) — честный выход из угадывания. На споте с единственным верным действием случайный клик иначе давал бы ~50% ложных GOOD и растягивал интервал невыученной карты. reveal раскрывает ответ и форсит AGAIN. Кнопка Easy убрана из UI (примитив EASY и эндпоинт /upgrade_easy оставлены в ядре, просто не вызываются).

3. SM-2 — формулы и константы

Рейтинги: `AGAIN=1`, `HARD=2`, `GOOD=3`, `EASY=4`. Бинарный ввод Learn порождает только **GOOD** или **AGAIN** (`EASY` — лишь через опциональный апгрейд, `HARD` не используется). `update_card(card, rating)` мутирует `(interval_days, ease_factor)` и пишет запись в `history` до мутации.

Рейтинг	interval_days	ease_factor	прочее
AGAIN · 1	сброс → 1	<code>max(1.3, ease - 0.20)</code>	<code>streak → 0</code>
GOOD · 3	лесенка <code>0→1→3→round(prev×ease)</code>	без изменений	<code>streak +1</code>
EASY · 4	<code>0→3</code> , иначе <code>round(prev×ease×1.3)</code>	<code>ease + 0.15</code>	<code>streak +1</code>
HARD · 2	<code>max(prev+1, int(prev×1.2))</code>	<code>max(1.3, ease - 0.15)</code>	в Learn UI не вызывается

Лесенка graduation для GOOD: `0 → 1 → 3 → 3×ease → ...` — впервые увидел → завтра; второй верный → через 3 дня; третий → через ~7-8 дней; далее геометрически `×ease`. `next_review = today + interval_days`.

Классификация: `classify()` → `pure_<action>`, если у доминирующего действия частота `≥ PURE_THRESHOLD`, иначе `mixed` (action-agnostic: `pure-fold` и `pure-open` читаются одинаково).

Константы (взяты из SuperMemo/Anki, не калиброваны под покер):

```
PURE_THRESHOLD = 0.95    DEFAULT_EASE = 2.5    MIN_EASE = 1.3
NEW_CARDS_PER_DAY = 15   LEARNED_THRESHOLD_DAYS = 21   (interval ≥ 21 → «в долговременной памяти»)
```

`upgrade_good_to_easy` (если позже жмут Easy на раскрытом экране): `interval = max(prev+1, round(prev×1.3))`, `ease += 0.15`, последняя запись `history` переписывается `GOOD→EASY`. Намеренно приблизительно: буферизировать pre-update состояние между двумя HTTP-вызовами дороже, чем стоит точность (~1 день расхождения на первом повторе).

4. Планировщик и очередь

`get_due_cards(cards, today, new_limit)` отдаёт в приоритете: (1) **повторы на сегодня** (`next_review ≤ today`) — всегда; (2) **новые карты**, капнутые на `new_limit` /день. Дневной кап считается из состояния (карта с `last_seen==today` и `total_seen==1` = введена сегодня), без отдельного поля.

Стабильный md5-шафл новых карт (поправка 2026-06-15, P-029). Новые карты вводятся не в порядке колоды (= порядок JSON, пары/топ-suited первыми), а в порядке `md5(card_id)` — детерминированно, не зависит от JSON-порядка, стабильно между днями. Иначе фиксированный дневной кап всегда срезал один и тот же верх диапазона и хоронил маргинальные руки границы. `new_limit` (10/15/30/50/∞) — кап показа очереди, не свойство колоды (в стейт не пишется).

API (`/api/srs`): `status` · `init` (идемпотентно, 409 без `force`) · `next` (карта без ответа) · `answer` (грейд + раскрытие + следующая) · `summary` · `reset`. `summarize()` даёт счётчики `total` / `new` / `due_today` / `young` / `learned` для дашборда.

5. Сознательные отклонения от канона SM-2

Что важно знать в первую очередь — где и почему мы отходим от учебникового SM-2/Anki.

Отклонение	Почему
Только 2 сигнала (GOOD/AGAIN), нет Hard и нет Again-как-самооценки	Ввод принудительный (клик действия), система объективно знает результат — самооценка поверх лишняя (решение #6).
Ось трудности мертва. При бинарном вводе GOOD никогда не двигает ease → ease заморожен на 2.5 для любой выученной карты	Следствие: pure AA (open 1.0) и противный близкий микс A5s (55/45) получают идентичную лесенку. Единственный регулятор SM-2 не работает (см. §7).
Нет потолка интервала в проде	SM-2 множит <code>xease</code> бесконечно (780, 1950 дней...). Для грайндера повторы «через годы» бессмысленны — но кап в SM-2 пока не введён (кандидат A.3).
«Правильно» = членство в стратегии, не бросок ГСЧ	Микс-карта верна при любом in-strategy действии (решение #7). То же в Practice Mode (решение #8, фикс P-018).
«Показать ответ» → AGAIN	Честное «не знаю» не должно маскироваться под верное угадывание (поправка #6).

6. FSRS-lite — экспериментальный параллельный трек

`srs_fsrs.py` — обёртка над `py-fsrs` (FSRS-6), **параллельно** SM-2 и **не подключённая** к `srs_api`. Цель — сравнить FSRS-планирование с нашим SM-2 на реальных данных и, если данные покажут систематический промах SM-2, переключить Learn. Импорт `fsrs` ленивый; пакета нет в `requirements.txt` (прод его не грузит).

6.1. Сидирование трудности — ключевой рычаг

При бинарном вводе ни наш `ease`, ни `difficulty` FSRS не двигаются на верном ответе. Единственный способ внести «эта карта трудная» — **засидировать** `difficulty` из частот стратегии до первого ответа:

```

dominant = max(strategy.values())
dominant ≥ 0.95          → D = 2.0                      (pure)
0.5 ≤ dominant < 0.95    → frac = (0.95 - dominant) / (0.95 - 0.5)
                        D = 4.0 + frac × (8.0 - 4.0)      → clamp [1, 10]
strategy пусто          → D = 5.0 (нейтрально)
Примеры: {open:1.0}→2.0 · {open:0.9}→~4.4 · {3bet:0.55}→~7.6 · {open:0.5}→8.0

```

Интуиция: чем труднее спот *запомнить*, тем чаще хотим его видеть. Pure-решение тривиально; близкий микс — самое трудное (надо помнить, что это микс, и его ~пропорцию). Конфиг планировщика: `desired_retention = 0.95`, `maximum_interval = 120`, `learning_steps = ()` (сразу к интервалам в днях, без кром-шагов на 1/10 мин).

6.2. Эмпирика интервалов (анализ-only, прод не тронут)

Прогон одной карты через планировщик с ответом ровно в день `due`; FSRS с выключенным фаззингом; для сравнения дёргается настоящий `srs.update_card`. Лесенка «всегда Good»:

Система	1	2	3	4	5	6	7	8	9
SM-2 (наша, ease 2.5, без капа)	1	3	8	20	50	125	312	780	1950
FSRS R=0.90 (дефолт)	2	11	46	163	497	1346	3292	7398	15466
FSRS R=0.95	1	3	8	19	43	89	175	324	575

Система	1	2	3	4	5	6	7	8	9
FSRS R=0.97	1	2	4	7	13	22	36	58	90

Наша SM-2-лесенка (×2.5) на первых шагах почти совпадает с FSRS при **R≈0.95** (1-3-8 → 1-3-8) — то есть SM-2 интуитивно настроен примерно на 95% удержания. Дефолтный FSRS R=0.90 для грайндера вреден (повторы на годы); SM-2 множит без потолка.

Различение трудности (тот же горизонт, R=0.95, интервал капнут на 120) — главный результат:

Карта	1	2	3	4	5	6	7	8	9
SM-2 — pure AA и mix A5s (одинаково!)	1	3	8	20	50	125	312	780	1950
FSRS D=2 — pure (AA RFI)	1	3	8	20	44	92	120	120	120
FSRS D=5 — средний микс (3bet 70/30)	1	2	5	10	19	35	62	105	120
FSRS D=8 — близкий микс (55/45)	1	2	3	5	7	11	16	23	32

Восстановление после ошибки (Good×3, Again, Good×2): SM-2 сбрасывает в 1 день, `ease 2.5→2.30`, и лезет вверх почти с прежней скоростью. FSRS R=0.95: difficulty прыгает `2.10→7.39`, последующие Good растут медленнее — FSRS «запоминает», что карта стала дырявой.

Ограничение бинарного ввода. При двух сигналах difficulty/ease фактически = «частота провалов». Карта, которую *случайно угадываешь* на близком миксе, выглядит для обеих систем «лёгкой». Поэтому сидирование difficulty по стратегии — не опция, а необходимость. Чтобы difficulty *доучивалась* сама, нужен сигнал богаче Again/Good — отдельное продуктивное решение, сейчас не делается.

7. Дорожная карта (Track A — логика SRS)

- **DONE A.0 / A.0.1 / A.5** — кнопка «Показать ответ»; md5-шафл + настраиваемый `new_limit`; auto-next 1.5 с.
- **В РАБОТЕ A.1 — history-логирование.** Поле `history: list[dict]` уже в модели; `update_card` пишет `{date, delta_days, rating, correct}` до мутации — это сырьё для будущего фита FSRS. Каждый день практики без логов = потерянные данные.
- **ПАССИВНО A.2 — накопление практики.** 2-3 недели реальных ответов; UX-неровности → `problems.md`. На 17.06 накоплено ~15+10 ответов за 1 день — фитить пока не на чём.
- **ОЖИДАЕТ ДАННЫХ A.3 — калибровка SM-2-констант** (`ease`, `MIN_EASE`, кап интервала, частота микс-карт) по данным A.1+A.2, не наугад.
- **ОТЛОЖЕНО (#11) A.4 — FSRS-апгрейд.** Добавить ось difficulty в `Card`. Каркас FSRS-lite (`srs_fsrs.py` + 45 тестов + A/B-харнесс `tools/fsrs_ab_compare.py`) **уже собран**, но под замком.

Под замком (решение #11), не делать без сигнала «SM-2 систематически промахивается на моих данных»: фит собственных параметров FSRS (`fsrs-optimizer`) до 2-3 недель данных; подключение FSRS-lite к `srs_api` (живой Learn на FSRS); полный SSP-MMC с матрицей policy. На нашем масштабе (~1500 карт, 1 игрок, бинарный ввод) ROI низкий. Переносимое из первоисточника = живая трудность (сидируется из стратегии) + целевое удержание ≈0.95 + потолок ≈120 дней — это и несёт FSRS-lite.

8. Честные ограничения анализа

- Дефолтные параметры FSRS-6 откалиброваны на ~220 млн повторов **английских слов** (MaiMemo). Форма кривой забывания на покерных диапазонах может отличаться — это допущение, не факт.
- Difficulty в таблице §6.2 задана вручную (синтетически), чтобы показать механизм. Реальная привязка «стратегия → difficulty» — работа A.4, её ещё предстоит выверить.
- Сравнение *интервалов* ≠ сравнение *качества запоминания*. Чтобы доказать, что FSRS учит лучше на нашем материале, нужны накопленные логи + оффлайн-оценка удержания (A.2+A.3).
- Капы и retention `0.95 / 120` — стартовая гипотеза, а не выверенный оптимум.
- Сама статья KDD'22 (Appendix A.2) предписывает в холодном старте использовать SM-2 → наш SM-2 не ошибка, а ровно стартовая фаза; разрыв в том, что у нас пока нет механизма из неё «вырасти».

9. Источники и референсы

Чем руководствовался при построении блока — канонические алгоритмы и первоисточник, плюс внутренние доки проекта.

- **SM-2 (ядро планировщика)**. P. Woźniak, SuperMemo (1987/1990): начальный `EF = 2.5`, интервал = `prev × EF`, EF падает на провале. Откуда наши `DEFAULT_EASE=2.5`, `MIN_EASE=1.3`. — supermemo.com/en/archives1990-2015/english/ol/sm2
- **Anki-конвенции**. Лимит новых карт/день, graduation-интервалы, «mature» при `interval ≥ 21` дн. Откуда `NEW_CARDS_PER_DAY=15`, `LEARNED_THRESHOLD_DAYS=21`, форма лесенки. — docs.ankiweb.net
- **Объективный бинарный грейдинг**. Chessable / Duolingo-философия: система оценивает по результату, без самооценки сложности. Основание решения #6.
- **FSRS / py-fsrs (эксперимент A.4)**. Open Spaced Repetition, пакет `fsrs` (FSRS-6, у нас 6.3.1). Растёт из DHP-модели MaiMemo (вариант DSR-модели Woźniak): `difficulty / stability / retrievability`. — github.com/open-spaced-repetition/py-fsrs · pypi.org/project/fsrs
- **Первоисточник FSRS (трек апгрейда A.4, не текущее ядро)**. Junyao Ye, Jingyong Su (HIT Shenzhen), Yilong Cao (MaiMemo). «*A Stochastic Shortest Path Algorithm for Optimizing Spaced Repetition Scheduling*», ACM SIGKDD '22. DOI `10.1145/3534678.3539081`. Модель памяти DHP + планировщик SSP-MMC (оптимизация стоимости повторов). Русский конспект — [docs/ssp_mmc_ru.md](#).
- **Внутренние документы**. [docs/fsrs_analysis.md](#) (анализ интервалов + §0 сравнение методик); [CLAUDE.md](#) решения #6 / #7 / #8 / #9 / #11; [docs/roadmap.md](#) Track A. Код: `srs.py`, `srs_api.py`, `srs_fsrs.py`, воспроизводимые скрипты `tools/fsrs_interval_sim.py`, `tools/fsrs_ab_compare.py`.

Примечание: числовые таблицы §6.2 получены нашим воспроизводимым прогоном (`tools/fsrs_interval_sim.py`, fsrs 6.3.1, фаззинг off, `learning_steps=()`) и могут отличаться от «голых» дефолтов FSRS из публичных источников из-за нашего конфига (retention/cap/steps).